FinEx

Combined Implementation Handbook

Equal-opportunity execution + free-tier scale-ready specification

React Web + Flutter Mobile + FastAPI + Lakebase/Postgres + Databricks Delta + AI/ML

Version 1.0 | 10 August 2026

Prepared for 12 rotating developers and one Business Analyst/UAT lead

Operating rule: keep the clear S1-PRD/API/DAT/AI task structure and owner rotation, but execute every card against the Free-Tier handbook's architecture, contracts, Definition of Ready/Done, deployment and safety requirements.

1. How the two handbooks are combined

- Execution spine - Equal-Opportunity Rotation Handbook v4: sprint names, task IDs, individual owners, reviewer paths, blocker guidance and the two-sprints-per-track rotation remain the developer-facing plan.
- Implementation depth - Free-Tier Scale-Ready Handbook v5: architecture, repository boundaries, OpenAPI/generated clients, Lakebase vs Delta responsibilities, provider adapters, deployment, Definition of Ready/Done, privacy, AI evaluation and free-tier guardrails become mandatory requirements on those cards.
- The confusing A-S1-UI / B-S1-API / G-S1-DATA work-package naming is not used on the team board. Its engineering detail is translated into the clearer S1-PRD-01-style cards.
- Conflict rule: sprint sequencing/ownership follows the Equal-Opportunity plan; storage/deployment/contract architecture follows the Free-Tier plan. Therefore operational transactional records use Lakebase/Postgres even where the older task wording says Delta; Delta remains the analytics/history/event/evaluation layer.

Why this is the recommended combination

- The team gets one straightforward card per person per sprint, with a visible outcome and reviewer.
- Management retains the detailed implementation controls needed for a scale-ready prototype and later paid-tier migration.
- No developer has to read a 100-page handbook to understand today's assignment; the detailed rules are embedded into each sprint card and shared checklists.

2. Canonical free-tier / scale-ready architecture

Area	Free-tier implementation	Scale-ready rule
React web	Azure Static Web Apps Free	Same React app can move to Standard/custom domain without frontend rewrite.
Flutter mobile	Android APK via GitHub Releases; iOS-compatible source/simulator	Store publication is deferred from zero-cost phase.
FastAPI	Dockerized API on Azure Container Apps Consumption	Scale replicas/resources later; API image and contracts stay stable.
Container registry	GitHub Container Registry	Move immutable images to Azure Container Registry later if needed.
Operational database	Databricks Lakebase Postgres Free Edition	Users, consent, chat metadata, portfolios, paper ledger, alerts, goals, spending; Alembic/SQLAlchemy remain portable.
Analytics / data engineering	Databricks Free Edition Delta + SQL	History, snapshots, events, reconciliation, features and evaluation data.
AI / RAG	Databricks model/foundation access + Vector Search + MLflow where available	LLMProvider/Retriever interfaces allow later paid endpoints.
Authentication	Google OAuth + backend JWT	Enterprise identity can be added later without changing client auth contract.
Notifications	In-app notifications + FCM no-cost foundation	Email/SMS provider can be added behind NotificationProvider later.
Market data	Prototype yfinance adapter + cache + mock adapter	Never claim guaranteed real-time; swap to licensed provider behind MarketDataProvider.
CI/CD	GitHub Actions	Contract generation, tests, container build and deployment.
Secrets	GitHub environment + Container Apps secrets	Move secret references to Azure Key Vault later; env names stay stable.

Provider boundary rule: domain services depend on operational_store, analytics_store, market_data_provider, llm_provider, retriever, notification_provider and object_store interfaces - not directly on vendor SDKs.

3. GitHub board workflow and acceptance rules

Your board	Detailed state	Meaning / rule
Backlog	DRAFT / NOT READY	Task may exist, but production implementation must not start while contracts, predecessors, examples or reviewers are missing.
Ready	READY	Combined Definition of Ready is complete; owner may start the production task.
In progress	IN PROGRESS / BLOCKED	Owner has a branch and daily evidence. If blocked, execute the listed mock/test/documentation fallback and record the blocking task ID.
In review	CONSUMER REVIEW / UAT	Producer says complete; technical consumer verifies the contract, then Syona verifies user-facing acceptance.
Done	ACCEPTED	Producer, consumer and UAT approvals are recorded; successor may start.

Combined Definition of Ready

- User story and business rules are approved.
- Required/optional fields, enumerations, units, time zones and currency rules are defined.
- OpenAPI, event, operational/analytics data and AI schemas required by the card are frozen for the sprint.
- Normal, empty, boundary, permission, provider-failure and validation examples exist.
- Generated mock server and TypeScript/Dart clients compile where the task has a client/API contract.
- Predecessor task IDs are Accepted, or the approved mock/fixture path is documented.
- Reviewer, downstream consumer and UAT owner are named.
- No unresolved issue changes the contract or deterministic financial calculation.

Combined Definition of Done

- Code/notebook/migration is merged after peer review.
- Required unit, contract, integration, UI-state, data-quality and AI-evaluation tests pass.
- No secret, token, private user data or sensitive prompt appears in source or logs.
- Downstream consumer validates output using the supplied sample data / generated contract.
- Handoff lists artifacts, version, run commands, sample payload/seed location, known limitations and newly unblocked tasks.
- Syona accepts user-visible behavior or records an approved exception.
- Deployment/demo evidence is attached whenever a runtime component changes.
- Owner can explain the problem, callers/dependencies, storage, failure modes, proving test and rollback path.

4. Week 0 - combined enablement and platform bootstrap

Week 0 combines the Equal-Opportunity common bootcamp with the Free-Tier handbook's concrete bootstrap ownership. Production Sprint 1 work stays in Backlog until the Week 0 exit gate is accepted.

Day	Everyone completes	Evidence
Day 1	Git flow, task IDs, branch/PR and code review; read specifications/contracts; clone/branch/commit/rebase/open PR/resolve conflict.	One reviewed documentation/training PR.
Day 2	Run React/Vite and Flutter locally; call a	Screenshots plus React/Flutter test output.

Day	Everyone completes	Evidence
	mock API; render loading/error/success; typed form + widget/component test.	
Day 3	FastAPI route/schema/service/repository, Pydantic validation, standardized envelope and pytest.	One GET/POST sample endpoint and passing tests.
Day 4	Lakebase/Postgres SQL + Alembic, Delta/Databricks notebook and at least two DQ assertions.	Migration, notebook export and query/DQ evidence.
Day 5	RAG basics, structured AI output, prompt versioning, golden-set evaluation, AI-assisted coding verification, Docker/deployment overview.	Validated mock AI response + mini evaluation/AI-usage note.

Individual bootstrap ownership

Owner	Task ID	Deliverable
Khyati	W0-KHY	Frontend/web route shell, design tokens, protected route placeholder and web commands documentation.
Nayan	W0-NAY	backend/app/ports interfaces for LLM, retriever, market, operational and analytics stores.
Nilisha	W0-NIL	Reusable React Button/Input/Card/Modal/Loading/Error components and /ui-kit route.
Payal	W0-PAY	AI JSON schema validator sample and five golden safety prompts.
Shivraj	W0-SHI	Flutter project flavors, navigation shell, theme and build instructions.
Mitali	W0-MIT	Databricks bundle skeleton, managed schemas, sample Delta job and DQ test.
Disha	W0-DIS	FastAPI factory, /health/live, /health/ready, error middleware and tests.
Pranjal	W0-PRA	Prompt registry format, mock LLM adapter and structured answer example.
Likhita	W0-LIK	Flutter reusable widgets and loading/empty/error screens.
Palak	W0-PAL	Lakebase/Postgres SQLAlchemy connection, Alembic baseline and transaction test.
Piyush	W0-PIY	OpenAPI validation, TypeScript/Dart client generation commands and mock server workflow.
Mrugank	W0-MRU	Evaluation runner, JSON report and baseline retrieval fixtures.
Syona	W0-SYO	Glossary, feature-spec/acceptance templates, dependency board, severity definitions and Definition of Ready checklist.

Week 0 exit gate

- Every developer can run React, Flutter, FastAPI, tests and a database migration.
- A sample vertical slice travels from web/mobile mock to FastAPI to Lakebase and returns a typed response.
- OpenAPI client generation and mock server work on a clean checkout.
- Databricks bundle validation succeeds and a sample job writes a test Delta table.
- No developer begins Sprint 1 with unresolved repository/tool access problems.

5. Eight-sprint execution roadmap

Sprint	Outcome	Alpha	Beta	Gamma	Delta
1	Platform foundation	Product	API	Data	AI/ML
2	Identity, onboarding and risk profile	API	Data	AI/ML	Product
3	Market data, watchlist and portfolio	Data	AI/ML	Product	API
4	AI financial assistant and RAG	AI/ML	Product	API	Data
5	Paper trading	Product	API	Data	AI/ML
6	Smart alerts and conversational portfolio intelligence	API	Data	AI/ML	Product
7	Goals and spending trend analysis	Data	AI/ML	Product	API
8	Production hardening and release	AI/ML	Product	API	Data

Developer	Cohort	S1	S2	S3	S4	S5	S6	S7	S8
Khyati	Alpha	Product	API	Data	AI/ML	Product	API	Data	AI/ML
Mitali	Alpha	Product	API	Data	AI/ML	Product	API	Data	AI/ML
Payal	Alpha	Product	API	Data	AI/ML	Product	API	Data	AI/ML
Nilisha	Beta	API	Data	AI/ML	Product	API	Data	AI/ML	Product
Nayan	Beta	API	Data	AI/ML	Product	API	Data	AI/ML	Product
Disha	Beta	API	Data	AI/ML	Product	API	Data	AI/ML	Product
Shivraj	Gamma	Data	AI/ML	Product	API	Data	AI/ML	Product	API
Pranjal	Gamma	Data	AI/ML	Product	API	Data	AI/ML	Product	API
Piyush	Gamma	Data	AI/ML	Product	API	Data	AI/ML	Product	API
Likhita	Delta	AI/ML	Product	API	Data	AI/ML	Product	API	Data
Palak	Delta	AI/ML	Product	API	Data	AI/ML	Product	API	Data
Mrugank	Delta	AI/ML	Product	API	Data	AI/ML	Product	API	Data

6.1 Sprint 1 - Platform foundation

Objective: Create common platform contracts, runnable application shells, Databricks foundation and AI evaluation baseline.

Syona / BA-UAT lane: Finalize user journey, UI wording, financial disclaimer placeholders and accept platform contracts. Evidence: journey map, acceptance checklist and signed S1 gates.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S1-PRD-01 Khyati	Create the React application shell: routes /login, /onboarding, /dashboard, /portfolio, /chat, /paper-trading, /alerts, /goals and /spending; add protected- route placeholder and shared layout.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Prepare the React shell for Azure Static Web Apps previews and Flutter dev/test/demo flavors; work against generated mocks before live APIs.	START: Day 1 after repository and branch rules are published in W0-ENG-01. IF BLOCKED: Create route map and wireframes while API contracts are pending.	EVIDENCE: PR containing src/app/router.tsx, AppShell, route tests, and screenshots at 360px and 1440px. DONE WHEN: All routes render without console errors; direct refresh works; protected routes redirect to /login using mock auth. REVIEW: Nilisha + Syona
S1-PRD-02 Mitali	Create the Flutter application shell with dev/staging/prod flavors, navigation routes matching the React route map, shared theme, AppButton, AppTextField, AppCard and LoadingView.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Prepare the React shell for Azure Static Web Apps previews and Flutter dev/test/demo flavors; work against generated mocks before live APIs.	START: Start after S1-PRD-01 publishes the route names; do not wait for finished React styling. IF BLOCKED: Install Flutter, complete two widget tutorials, and build widgets against static data.	EVIDENCE: Flutter PR with lib/core/theme, lib/core/router, lib/shared/widgets, widget tests and Android debug build evidence. DONE WHEN: flutter analyze has zero errors; app opens on Android emulator; every shared widget has loading/disabled/error example. REVIEW: Shivraj + Khyati
S1-PRD-03 Payal	Create the cross-platform UI contract: field names, button labels, validation message wording, loading/empty/error states and accessibility checklist for the common routes.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Prepare the React shell for Azure Static Web Apps previews and Flutter dev/test/demo flavors; work against generated mocks before live APIs.	START: Start with the approved product glossary from Syona; finalize after S1-PRD-01 and S1-PRD-02 expose components. IF BLOCKED: Draft test cases and mock data while shells are being built.	EVIDENCE: docs/ui-contract-v1.md, mock JSON fixtures, React smoke tests and Flutter widget smoke tests. DONE WHEN: Each route has a documented success, loading, empty and error state; wording is identical unless platform-specific. REVIEW: Syona + Khyati

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S1-API-01 Nilisha	Create FastAPI project factory, /api/v1/health/live, /api/v1/health/ready, configuration loader and dev/staging/prod settings validation.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Containerize FastAPI for Azure Container Apps Consumption, publish via GHCR, and make liveness/readiness fail safely.	START: Day 1 after repository structure is published. IF BLOCKED: Create endpoint skeleton and tests with an in-memory configuration.	EVIDENCE: backend/app/factory.py, config.py, health router, pytest tests and .env.example without secrets. DONE WHEN: Application fails clearly when required settings are absent; health tests return expected status and schema. REVIEW: Palak + Disha
S1-API-02 Nayan	Define API v1 conventions: response envelope, ApiError schema, validation error format, pagination, correlation ID and OpenAPI examples; implement middleware.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Containerize FastAPI for Azure Container Apps Consumption, publish via GHCR, and make liveness/readiness fail safely.	START: Start after S1-API-01 app factory can register middleware. IF BLOCKED: Draft the API standard document and example JSON before middleware is ready.	EVIDENCE: docs/api-standard-v1.md, Pydantic response/error models, correlation middleware and unit tests. DONE WHEN: A sample endpoint returns correlation ID and standardized 200, 422 and 500 payloads; React/Flutter reviewers approve the schema. REVIEW: Palak + Khyati + Shivraj
S1-API-03 Disha	Set up backend quality gates: Ruff/formatting, type checking, pytest coverage, secret scan and CI workflow; create auth dependency stub used by protected sample route.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Containerize FastAPI for Azure Container Apps Consumption, publish via GHCR, and make liveness/readiness fail safely.	START: Start after S1-API-01 creates the backend package. IF BLOCKED: Prepare CI YAML and lint configuration against a tiny sample module.	EVIDENCE: CI config, requirements/pyproject updates, protected sample route, tests for missing/invalid mock token. DONE WHEN: A deliberately failing lint/test blocks CI; successful run is attached to the sprint evidence. REVIEW: Palak + Payal

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S1-DAT-01 Shivraj	Create Databricks catalog and schema design for dev/staging/prod, naming rules and ownership map; draft DDL for users, portfolios and system events.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Establish logical dev/test/demo schemas. Where the older task says operational Delta tables, use Lakebase for operational data and Delta for analytical copies/events.	START: Day 1 after Databricks workspace access is confirmed. IF BLOCKED: Use local SQL files and a data dictionary if workspace access is delayed.	EVIDENCE: data/ddl/001_core.sql, docs/data-naming.md and catalog/schema creation notebook. DONE WHEN: DDL is repeatable, uses explicit types, includes created_at/updated_at and passes peer review. REVIEW: Mitali + Palak
S1-DAT-02 Pranjal	Implement Delta tables for users, user_profiles, portfolios, portfolio_holdings and application_events with seed data notebook.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Establish logical dev/test/demo schemas. Where the older task says operational Delta tables, use Lakebase for operational data and Delta for analytical copies/events.	START: Start after S1-DAT-01 freezes table names and required columns. IF BLOCKED: Create sample Pandas schemas and fixture CSV files before tables exist.	EVIDENCE: Databricks notebook 01_create_core_tables, seed notebook and sample query output. DONE WHEN: Notebooks can run twice without duplicate schema errors; seed rows can be queried through SQL Warehouse. REVIEW: Mitali + Shivraj
S1-DAT-03 Piyush	Create data quality checks for non-null IDs, valid timestamps, unique emails, positive holding quantity and portfolio ownership; publish a basic SQL monitoring query.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Establish logical dev/test/demo schemas. Where the older task says operational Delta tables, use Lakebase for operational data and Delta for analytical copies/events.	START: Start after S1-DAT-02 creates tables; write test specifications first. IF BLOCKED: Build checks against local fixtures while waiting.	EVIDENCE: notebooks/02_core_data_quality, expected-results table and screenshot/query evidence. DONE WHEN: Checks identify intentionally bad rows and pass for clean seed data; failures include table, rule and offending count. REVIEW: Mitali + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S1-AI-01 Likhita	Write the FinEx AI use-case and safety specification: supported intents, prohibited claims, answer template, disclaimers, data freshness and citation requirements.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Create swappable LLM/retriever adapters, a mock path for quota limits, and the initial golden evaluation baseline.	START: Day 1 after Syona publishes the product glossary and legal wording placeholders. IF BLOCKED: Review 30 sample finance questions and classify allowed, caution and refused responses.	EVIDENCE: docs/ai-answer-policy-v1.md and a labeled CSV with at least 30 prompts. DONE WHEN: Every prompt has expected intent, required sources, disclaimer and pass/fail criteria. REVIEW: Nayan + Syona
S1-AI-02 Palak	Create a Databricks RAG experiment notebook that ingests three approved sample financial documents, chunks text, creates embeddings through the selected test model and stores retrievable records.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Create swappable LLM/retriever adapters, a mock path for quota limits, and the initial golden evaluation baseline.	START: Start after S1-DAT-01 publishes the AI schema naming rule and S1-AI-01 defines citation metadata. IF BLOCKED: Prototype chunking locally with plain text files if model endpoint is not ready.	EVIDENCE: notebooks/ai/01_rag_prototype, sample document registry and retrieval output for five questions. DONE WHEN: Each retrieved chunk includes source, document date, section and chunk ID; notebook is reproducible. REVIEW: Nayan + Pranjal
S1-AI-03 Mrugank	Build AI evaluation harness v0 with exact-match/keyword checks for education answers, citation presence, disclaimer presence, latency and manual rating fields.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Create swappable LLM/retriever adapters, a mock path for quota limits, and the initial golden evaluation baseline.	START: Start after S1-AI-01 publishes labeled prompts; retrieval metrics may be added after S1-AI-02. IF BLOCKED: Create the scoring script against mocked responses first.	EVIDENCE: ai/evaluation/evaluate.py, evaluation CSV template and baseline report. DONE WHEN: Script produces per-case results and aggregate scores; failed cases are traceable to prompt ID. REVIEW: Nayan + Payal

Sprint exit gate

- React, Flutter and FastAPI start locally.
- Core Delta tables are queryable.
- API and AI standards are versioned.
- CI and evaluation baseline pass.

6.2 Sprint 2 - Identity, onboarding and risk profile

Objective: Deliver secure identity, onboarding, consent and explainable risk profiling.

Syona / BA-UAT lane: Approve onboarding questions, consent wording, risk questionnaire/scoring boundaries and run UAT. Evidence: questionnaire v1, UAT evidence and defects.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S2-PRD-01 Likhita	Build React login, consent, onboarding and risk questionnaire screens using the shared UI contract; integrate auth/profile endpoints when available.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Use approved token storage; onboarding must resume and consent/risk fields/errors must match on React and Flutter.	START: Start UI scaffolding after S1-PRD-01; connect APIs only after S2-API-01 and S2-API-02 pass contract tests. IF BLOCKED: Use mocked service responses while backend is pending.	EVIDENCE: React pages, typed services, validation tests and screenshots. DONE WHEN: User can resume incomplete onboarding; required consent is explicit; API and UI validation messages match. REVIEW: Khyati + Syona
S2-PRD-02 Palak	Build Flutter login, consent, onboarding and risk questionnaire flow with secure token storage and resume behavior.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Use approved token storage; onboarding must resume and consent/risk fields/errors must match on React and Flutter.	START: Start after S1-PRD-02; integrate after S2-API-01 and S2-API-02 are available in staging. IF BLOCKED: Implement against mock repository and local token store first.	EVIDENCE: Flutter feature modules, widget tests and emulator demo recording. DONE WHEN: Fresh install, returning user and expired-token flows work; no token is logged. REVIEW: Shivraj + Syona
S2-PRD-03 Mrugank	Create cross-platform onboarding test automation and analytics events for start, step completion, validation failure, risk completion and abandonment.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Use approved token storage; onboarding must resume and consent/risk fields/errors must match on React and Flutter.	START: Start after S2-PRD-01/02 publish stable test IDs and S1-PRD-03 event naming rules. IF BLOCKED: Prepare event dictionary and manual cases before UI completion.	EVIDENCE: React tests, Flutter integration tests, event specification and evidence from test event table. DONE WHEN: Event payload contains no answers that reveal sensitive financial details; same logical event names are used on both platforms. REVIEW: Payal + Mitali + Syona

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S2-API-01 Khyati	Implement /api/v1/auth/google/callback, JWT creation/validation, refresh/logout and /api/v1/users/me using the shared error envelope.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use Google OAuth + backend JWT/refresh/logout. Risk scores are deterministic; AI only explains them.	START: Start after S1-API-01, S1-API-02 and S1-DAT-02 are complete. IF BLOCKED: Build JWT service and tests using a mock Google identity payload while OAuth credentials are pending.	EVIDENCE: auth router/service/repository, token tests, OpenAPI examples and Postman/HTTP evidence. DONE WHEN: Valid mock identity creates or loads a user; expired/revoked tokens return standardized 401; logout invalidates refresh token. REVIEW: Palak + Disha
S2-API-02 Mitali	Implement onboarding and consent endpoints: GET/PUT /users/me/profile, POST /onboarding/complete, GET /risk/questions and POST /risk/assessments.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use Google OAuth + backend JWT/refresh/logout. Risk scores are deterministic; AI only explains them.	START: Start after S2-DAT-01 and S2-DAT-02 freeze columns; route skeleton may begin after S1-API-02. IF BLOCKED: Write Pydantic schemas and unit tests with repository mocks while tables are pending.	EVIDENCE: routers/onboarding.py, schemas, services, repository methods and tests. DONE WHEN: Invalid answer IDs are rejected; questionnaire version is REVIEW:
S2-API-03 Payal	Create contract and integration tests for authentication, profile, consent and risk endpoints; add fixtures for new, returning, expired-token and incomplete-profile users.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use Google OAuth + backend JWT/refresh/logout. Risk scores are deterministic; AI only explains them.	START: Start once S2-API-01 and S2-API-02 expose stable schemas. IF BLOCKED: Prepare test matrix and fixtures from API specification before implementation finishes.	EVIDENCE: pytest integration suite, test data reset script and endpoint coverage report. DONE WHEN: All critical paths pass; each validation/error code has at least one test; reset script restores clean state. REVIEW: Disha + Syona

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S2-DAT-01 Nilisha	Create Delta tables users, user_profiles, user_preferences, user_consents and auth_audit_events, including merge/upsert notebook.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Identity, consent, onboarding drafts and risk assessments stay operational in Lakebase; only approved audit/trend copies go to Delta, with versions preserved.	START: Start after S1-DAT-01 naming standard; align user identifiers with S2-API-01 before finalizing. IF BLOCKED: Draft DDL and sample rows while auth code is in progress.	EVIDENCE: data/ddl/010_identity.sql and notebook 10_identity_upsert. DONE WHEN: One user can have versioned consent records; email uniqueness and audit timestamps are enforced by checks. REVIEW: Mitali + Palak
S2-DAT-02 Nayan	Create versioned risk_questionnaires, risk_questions, risk_assessments and risk_assessment_answers tables plus a notebook to load questionnaire v1.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Identity, consent, onboarding drafts and risk assessments stay operational in Lakebase; only approved audit/trend copies go to Delta, with versions preserved.	START: Start after Syona approves questionnaire v1 and S1- DAT-01 naming standard. IF BLOCKED: Prepare normalized CSV/JSON questionnaire file while approval is pending.	EVIDENCE: DDL, questionnaire seed file and loader notebook. DONE WHEN: Question order, scoring weight and version are queryable; historical assessments remain linked to original version. REVIEW: Mitali + Syona
S2-DAT-03 Disha	Create identity/risk data quality checks and an audit query showing profile completeness, consent status, duplicate identity and orphan risk answers.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Identity, consent, onboarding drafts and risk assessments stay operational in Lakebase; only approved audit/trend copies go to Delta, with versions preserved.	START: Start after S2-DAT-01 and S2-DAT-02 tables exist. IF BLOCKED: Write rule definitions and expected bad- row fixtures first.	EVIDENCE: notebook 12_identity_risk_dq and SQL dashboard query. DONE WHEN: Injected duplicate/orphan rows are detected; clean test dataset returns zero critical failures. REVIEW: Piyush + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S2-AI-01 Shivraj	Implement deterministic risk scoring v1 from approved questionnaire weights, separating willingness and capacity scores and mapping to conservative/moderate/aggressive.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Separate deterministic scoring from explanation and provide a safe non-LLM fallback; test boundary profiles.	START: Start after S2-DAT-02 publishes questionnaire v1 and Syona signs scoring rules. IF BLOCKED: Build scoring function and unit tests from a local JSON questionnaire.	EVIDENCE: ai/risk/scoring.py, configuration file and tests for at least 12 profiles. DONE WHEN: Same answers always produce same score; boundary cases are documented; no LLM is used for numeric scoring. REVIEW: Nayan + Syona
S2-AI-02 Pranjal	Create prompt/template that explains the deterministic risk result in beginner language, cites influencing answers and includes limitations.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Separate deterministic scoring from explanation and provide a safe non-LLM fallback; test boundary profiles.	START: Start after S2-AI-01 returns structured score components and S1-AI-01 answer policy is approved. IF BLOCKED: Draft prompt and mocked output examples while scorer is being completed.	EVIDENCE: prompt file, response schema and 10 generated explanation examples. DONE WHEN: Explanation never changes numeric score, mentions top factors and does not promise returns. REVIEW: Nayan + Likhita
S2-AI-03 Piyush	Extend evaluation harness with risk-profile cases: score consistency, factor coverage, prohibited claims, readability and disclaimer checks.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Separate deterministic scoring from explanation and provide a safe non-LLM fallback; test boundary profiles.	START: Start after S2-AI-01 and S2-AI-02 produce callable functions or fixtures. IF BLOCKED: Create expected cases and manual rating rubric beforehand.	EVIDENCE: risk evaluation dataset, automated checks and baseline report. DONE WHEN: All deterministic scores match expected values; explanation safety pass rate is reported separately. REVIEW: Mrugank + Syona

Sprint exit gate

- New and returning users complete auth/onboarding on web and mobile.
- Risk score is deterministic and explanation passes evaluation.
- Consent/audit data is queryable.

6.3 Sprint 3 - Market data, watchlist and portfolio

Objective: Deliver market data, watchlists and deterministic portfolio views.

Syona / BA-UAT lane: Approve market-data freshness wording, portfolio metrics and watchlist scenarios; query DQ dashboard. Evidence: acceptance cases and SQL evidence.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S3-PRD-01 Shivraj	Build React market search, watchlist and portfolio dashboard: quote freshness badge, holdings table, allocation chart, loading/empty/error states.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Show source/freshness explicitly and never call prototype data guaranteed real-time. Portfolio totals come from backend metrics.	START: Start layout after S1-PRD-01; bind live data only after S3-API-01 and S3-API-02 pass tests. IF BLOCKED: Use typed fixtures and mock service worker while APIs are pending.	EVIDENCE: React feature pages, services, tests and accessibility screenshots. DONE WHEN: User can add/remove symbol, view freshness, and see portfolio totals without client-side financial calculations. REVIEW: Khyati + Syona
S3-PRD-02 Pranjal	Build Flutter market search, watchlist and portfolio dashboard with the same fields and states as React.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Show source/freshness explicitly and never call prototype data guaranteed real-time. Portfolio totals come from backend metrics.	START: Start after S1-PRD-02 and S1-PRD-03; integrate after S3-API-01/02 staging availability. IF BLOCKED: Use repository mocks before API access.	EVIDENCE: Flutter feature modules, widget/integration tests and Android demo. DONE WHEN: Cross-platform field and validation parity checklist passes. REVIEW: Shivraj + Syona
S3-PRD-03 Piyush	Implement accessible chart wrappers, API response mapping tests and end-to- end watchlist/portfolio flows for React and Flutter.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Show source/freshness explicitly and never call prototype data guaranteed real-time. Portfolio totals come from backend metrics.	START: Start chart work from fixtures; start E2E after S3-PRD- 01/02 and APIs are integrated. IF BLOCKED: Prepare test IDs, expected datasets and text summaries first.	EVIDENCE: Chart components, test suites and data reconciliation evidence. DONE WHEN: Chart labels match API values; watchlist change persists after refresh; portfolio UI equals API snapshot. REVIEW: Payal + Mitali

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S3-API-01 Likhita	Implement /market/search, /market/quotes/{symbol}, /watchlists and watchlist item CRUD with cache/freshness fields.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use a replaceable MarketDataProvider (prototype yfinance + cache + mock), normalize symbols, expose freshness, and never convert missing price to zero.	START: Start schemas after S1-API-02; repository integration after S3-DAT-01. IF BLOCKED: Develop endpoints against repository mocks first.	EVIDENCE: routers/market.py, watchlists.py, service/repository code and tests. DONE WHEN: Unsupported symbol, duplicate watchlist item, stale provider and provider failure return documented outcomes. REVIEW: Palak + Nilisha
S3-API-02 Palak	Implement portfolio CRUD, holdings validation, snapshot calculation and /portfolios/{id}/summary with deterministic totals and allocations.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use a replaceable MarketDataProvider (prototype yfinance + cache + mock), normalize symbols, expose freshness, and never convert missing price to zero.	START: Start after core portfolio tables from S1-DAT-02; quote-dependent valuation waits for S3-DAT-02. IF BLOCKED: Build calculations with fixed quote fixtures first.	EVIDENCE: portfolio router/service, calculation module and unit/integration tests. DONE WHEN: Allocation and gain/loss calculations pass fixed expected cases; user cannot access another user's portfolio. REVIEW: Disha + Mitali
S3-API-03 Mrugank	Add caching, pagination, provider error mapping and contract tests for market/watchlist/portfolio endpoints.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Use a replaceable MarketDataProvider (prototype yfinance + cache + mock), normalize symbols, expose freshness, and never convert missing price to zero.	START: Start after S3-API-01/02 publish route signatures. IF BLOCKED: Prepare test matrix, fake provider and load fixtures early.	EVIDENCE: test suite, cache configuration and API contract report. DONE WHEN: Safe GET calls use cache; stale data is labeled; failures never masquerade as zero price. REVIEW: Payal + Khyati

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S3-DAT-01 Khyati	Create securities, market_quotes, market_price_history, watchlists and watchlist_items Delta tables with provider/freshness metadata.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Watchlists/portfolios are operational; market history and portfolio snapshots/metrics are Delta analytics. Record source, observed_at, ingested_at and freshness.	START: Start after S1-DAT-01; confirm required fields with S3- API-01 before final merge. IF BLOCKED: Draft DDL and sample market fixtures independently.	EVIDENCE: data/ddl/020_market.sql and table creation notebook. DONE WHEN: Each quote records source, observed_at, ingested_at and freshness class; watchlist uniqueness is testable. REVIEW: Mitali + Palak
S3-DAT-02 Mitali	Build market-data ingestion notebook/job: symbol normalization, provider adapter, retry, deduplication and MERGE into quote/history tables.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Watchlists/portfolios are operational; market history and portfolio snapshots/metrics are Delta analytics. Record source, observed_at, ingested_at and freshness.	START: Start after S3-DAT-01 creates tables; provider credentials can be replaced with recorded sample responses. IF BLOCKED: Implement adapter interface and parse fixture files first.	EVIDENCE: notebooks/market/20_ingest_quotes, provider adapter and run log table. DONE WHEN: Repeated input does not duplicate history; invalid symbols are quarantined; freshness label is calculated. REVIEW: Nayan + Palak
S3-DAT-03 Payal	Create market freshness and portfolio data quality checks: stale quote, duplicate quote, orphan watchlist item, invalid holding quantity and allocation sum.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Watchlists/portfolios are operational; market history and portfolio snapshots/metrics are Delta analytics. Record source, observed_at, ingested_at and freshness.	START: Start after S3-DAT-01 and existing portfolio tables are available. IF BLOCKED: Write SQL checks against fixture tables first.	EVIDENCE: DQ notebook, alert thresholds and sample failed-run report. DONE WHEN: Checks distinguish warning versus blocking failure and identify affected symbols/users. REVIEW: Piyush + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S3-AI-01 Nilisha	Create structured company/fund comparison prompt and response schema for returns, fees, risk, data date, assumptions and source list.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Comparison and portfolio narratives must use deterministic metrics/snapshots; each market-specific metric needs a date/source or unavailable state.	START: Start after S1-AI-01 and after S3-DAT-01 defines available market fields. IF BLOCKED: Build examples from static fixtures while live ingestion is pending.	EVIDENCE: prompt templates, JSON schema and 12 comparison examples. DONE WHEN: Output does not invent missing metrics; every metric includes date/source or 'not available'. REVIEW: Nayan + Syona
S3-AI-02 Nayan	Implement portfolio explanation tool that receives deterministic backend metrics and produces beginner-friendly concentration/diversification narratives.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Comparison and portfolio narratives must use deterministic metrics/snapshots; each market-specific metric needs a date/source or unavailable state.	START: Start after S3-API-02 defines portfolio metric response and S2-AI-02 establishes explanation pattern. IF BLOCKED: Use JSON fixtures generated from sample portfolios first.	EVIDENCE: tool interface, prompt, model call wrapper and sample outputs. DONE WHEN: No arithmetic is delegated to the LLM; narrative numbers exactly match tool input. REVIEW: Pranjal + Palak
S3-AI-03 Disha	Build factuality/evaluation set for comparisons and portfolio explanations, including missing-data, stale-data and contradictory-source cases.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Comparison and portfolio narratives must use deterministic metrics/snapshots; each market-specific metric needs a date/source or unavailable state.	START: Start after S3-AI-01/02 schemas are stable. IF BLOCKED: Draft 20 test cases and expected failure rules in parallel.	EVIDENCE: evaluation dataset and automated result report. DONE WHEN: Report separates citation, numeric consistency, freshness disclosure and prohibited-claim scores. REVIEW: Mrugank + Payal

Sprint exit gate

- Market freshness is visible.
- Watchlist and portfolio values reconcile with API/data.
- No missing quote is displayed as zero.

6.4 Sprint 4 - AI financial assistant and RAG

Objective: Deliver citation-based AI research chat backed by a versioned RAG pipeline.

Syona / BA-UAT lane: Create at least 40 representative AI questions, expected citations and manual rating rubric; run answer review. Evidence: evaluation dataset and scored sample.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S4-PRD-01 Nilisha	Build React AI chat: conversation list, composer, response states, citation cards, copy, feedback and delete confirmation.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Display citations, limitations, data-as-of and fallback states consistently; never send prompt text/tokens/private context to analytics.	START: Start UI with fixtures after S1-PRD-01; integrate only after S4-API-01/02 contract tests pass. IF BLOCKED: Use mocked streaming/non-streaming responses.	EVIDENCE: React chat feature, typed service, unit/E2E tests and accessibility report. DONE WHEN: Retry preserves prompt; citation opens metadata; duplicate send is blocked; errors do not erase history. REVIEW: Khyati + Syona
S4-PRD-02 Nayan	Build Flutter AI chat with history, citations, feedback, session expiry and safe rendering of long financial content.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Display citations, limitations, data-as-of and fallback states consistently; never send prompt text/tokens/private context to analytics.	START: Start from fixture repository after S1-PRD-02; connect after S4-API-01/02 staging readiness. IF BLOCKED: Develop widgets and state management using fake responses.	EVIDENCE: Flutter chat module, widget/integration tests and emulator evidence. DONE WHEN: State survives rebuild; long content does not overflow; citation fields match React. REVIEW: Shivraj + Syona
S4-PRD-03 Disha	Create cross-platform chat test suite and analytics: send, success, failure, citation click, feedback, delete and session expiry.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Display citations, limitations, data-as-of and fallback states consistently; never send prompt text/tokens/private context to analytics.	START: Start event/test design immediately; automation waits for S4-PRD-01/02 stable test IDs. IF BLOCKED: Prepare fixtures and expected events first.	EVIDENCE: React and Flutter automated tests, event dictionary update and sample event queries. DONE WHEN: No prompt content or token is included in analytics; all critical chat states are tested. REVIEW: Payal + Mitali

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S4-API-01 Shivraj	Implement conversation/session/message/feedback endpoints with user isolation and pagination.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. User-isolate conversations; orchestration retrieves authorized context, validates citations/output, retries once and fails safely rather than fabricating.	START: Start after S4-DAT-01 creates conversation/message tables and S2-API-01 auth is stable. IF BLOCKED: Create schemas and repository-mock tests first.	EVIDENCE: chat router/service/repository, OpenAPI examples and tests. DONE WHEN: Users can only access their own history; delete behavior is defined; pagination is deterministic. REVIEW: Palak + Disha
S4-API-02 Pranjal	Implement AI orchestration service: classify intent, retrieve context, call model serving, apply timeout/retry policy, invoke guardrails and return citation-rich schema.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. User-isolate conversations; orchestration retrieves authorized context, validates citations/output, retries once and fails safely rather than fabricating.	START: Start after S4-DAT-02 index query works, S4-AI-01 loads content and S1-API-02 response standard exists. IF BLOCKED: Build interfaces with fake retriever/model/guardrail clients first.	EVIDENCE: orchestration service, client adapters, unit/integration tests and tracing fields. DONE WHEN: Timeout returns safe error; each answer has model/version, data date and citations; no silent fallback to fabricated response. REVIEW: Nayan + Palak
S4-API-03 Piyush	Implement feedback endpoint, correlation tracing across API/retrieval/model calls and integration tests for education, comparison and refusal intents.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. User-isolate conversations; orchestration retrieves authorized context, validates citations/output, retries once and fails safely rather than fabricating.	START: Start after S4-API-01/02 expose identifiers and tracing hooks. IF BLOCKED: Prepare integration fixtures and expected trace fields.	EVIDENCE: feedback API, trace tests and contract evidence. DONE WHEN: One correlation ID links message, retrieval event and model call; feedback references the exact answer version. REVIEW: Payal + Mrugank

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S4-DAT-01 Likhita	Create document_registry, document_chunks, ai_conversations, ai_messages, ai_feedback and retrieval_events Delta tables.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Chat metadata is operational; document chunks, retrieval events and evaluation data are Databricks assets accessed through the retriever interface.	START: Start after S1-DAT-01 and S1-AI-01 metadata requirements. IF BLOCKED: Draft DDL and sample records first.	EVIDENCE: data/ddl/030_ai_rag.sql and creation notebook. DONE WHEN: Tables support document versioning, user isolation, citations, latency and model version fields. REVIEW: Mitali + Palak
S4-DAT-02 Palak	Create and populate the Databricks Vector Search index and retrieval workflow with metadata filters for company, document type, period and date.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Chat metadata is operational; document chunks, retrieval events and evaluation data are Databricks assets accessed through the retriever interface.	START: Start after S4-DAT-01 tables and S4-AI-01 initial chunks exist. IF BLOCKED: Define index configuration and test query script before data load.	EVIDENCE: index creation workflow, query helper and retrieval evidence. DONE WHEN: Top-k results respect filters; deleted/superseded documents are excluded. REVIEW: Nayan + Mitali
S4-DAT-03 Mrugank	Create AI data lineage and quality checks for missing metadata, duplicate chunks, orphan citations, failed embeddings and unlinked feedback.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Chat metadata is operational; document chunks, retrieval events and evaluation data are Databricks assets accessed through the retriever interface.	START: Start after S4-DAT-01/02 and S4-API-01 identifiers are stable. IF BLOCKED: Write rules and synthetic failure data in parallel.	EVIDENCE: DQ notebook, lineage query and failed-run report. DONE WHEN: All synthetic failures are detected; report identifies pipeline run and affected document/message. REVIEW: Piyush + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S4-AI-01 Khyati	Build document ingestion pipeline for approved filings/factsheets: registry entry, text extraction, section-aware chunking, metadata validation and embedding job.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Use versioned RAG ingestion, retrieval evaluation, structured answers, citation validation and guardrails with full trace versions.	START: Start after S4-DAT-01 creates registry/chunk tables and S1-AI-02 prototype is reviewed. IF BLOCKED: Develop chunker and metadata validator locally with sample PDFs/text.	EVIDENCE: production-style Databricks workflow, reusable Python modules and run evidence. DONE WHEN: Every chunk has source/date/section/page reference; invalid documents are quarantined with reason. REVIEW: Nayan + Mitali
S4-AI-02 Mitali	Implement retrieval evaluation notebook measuring recall@k, metadata filter correctness and stale-document exclusion on a labeled query set.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Use versioned RAG ingestion, retrieval evaluation, structured answers, citation validation and guardrails with full trace versions.	START: Start after S4-DAT-02 index is queryable and S4-AI-01 loads documents. IF BLOCKED: Create labeled queries and expected documents beforehand.	EVIDENCE: evaluation notebook, dataset and baseline metrics. DONE WHEN: Each query records expected source, retrieved sources and failure reason; metrics are reproducible. REVIEW: Nayan + Mrugank
S4-AI-03 Payal	Implement answer guardrails and citation validator: block unsupported numeric claims, require citations for document claims, add freshness/limitations/disclaimer.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Use versioned RAG ingestion, retrieval evaluation, structured answers, citation validation and guardrails with full trace versions.	START: Start after S4-API-02 defines model response envelope and S4-AI-01 metadata is stable. IF BLOCKED: Develop validator against mocked answers first.	EVIDENCE: guardrail module, tests and red-team cases. DONE WHEN: Answers failing citation/numeric checks are rejected or rewritten into a safe fallback; reasons are logged. REVIEW: Pranjal + Syona

Sprint exit gate

- Chat returns cited answers from approved documents.
- Conversation ownership and feedback work.
- Retrieval and safety metrics are recorded.

6.5 Sprint 5 - Paper trading

Objective: Deliver a reconciled paper-trading ledger and cross-platform simulation experience.

Syona / BA-UAT lane: Create paper-trading scenario matrix and verify disclaimers, ledger outcomes and reset behavior. Evidence: UAT/reconciliation sign-off.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S5-PRD-01 Khyati	Build React paper-trading screens: virtual account summary, market buy/sell order ticket, confirmation, positions, order history and visible simulation disclaimer.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Every screen states no real money is involved, shows quote freshness and reconciles balances/positions after refresh.	START: Start UI from fixtures after S3 market schemas; integrate after S5-API-01/02 contract tests pass. IF BLOCKED: Use deterministic price and account fixtures while backend is pending.	EVIDENCE: React feature module, service layer, form tests and E2E buy/sell tests. DONE WHEN: Invalid quantity/cash/holding cases cannot submit; duplicate clicks create one request; all screens label prices as live/delayed/latest. REVIEW: Nilisha + Syona
S5-PRD-02 Mitali	Build Flutter paper-trading account, order ticket, positions and history with the same validation and disclaimer as React.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Every screen states no real money is involved, shows quote freshness and reconciles balances/positions after refresh.	START: Start from mocks after S1-PRD-02 and S3 field contracts; connect after S5-API-01/02 staging. IF BLOCKED: Create widget fixtures and local validation first.	EVIDENCE: Flutter feature module, widget/integration tests and emulator demo. DONE WHEN: Cross-platform validation matrix passes; app recovers cleanly from price-unavailable response. REVIEW: Shivraj + Syona
S5-PRD-03 Payal	Create paper-trading E2E and reconciliation tests covering account creation, buy, partial sell, insufficient cash, excessive sell, stale price, duplicate request and reset.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Every screen states no real money is involved, shows quote freshness and reconciles balances/positions after refresh.	START: Start scenario design after S5-API-01 contract draft; automation after S5-PRD-01/02 and S5-API-02. IF BLOCKED: Prepare test users, expected ledger entries and API fixtures.	EVIDENCE: React/Flutter E2E suite, reconciliation sheet and defect report. DONE WHEN: UI balances, positions and backend ledger agree after every scenario; reset requires explicit confirmation. REVIEW: Mrugank + Syona

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S5-API-01 Nilisha	Implement paper account creation/reset, order submit/list/get and positions endpoints with idempotency key and user isolation.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Paper execution is atomic and idempotent across order, trade, cash ledger and position; no brokerage execution API.	START: Start after S5-DAT-01 creates tables, S2 auth is stable and S3 market quote service is available. IF BLOCKED: Build schemas and repository mocks before table completion.	EVIDENCE: paper_trading router/service/repository and tests. DONE WHEN: Unauthorized access is blocked; repeated idempotency key returns same result; reset behavior is audited. REVIEW: Palak + Disha
S5-API-02 Nayan	Implement deterministic execution/ledger service for market buy/sell: validate usable price, cash/quantity, create trade, cash ledger entry and updated position atomically.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Paper execution is atomic and idempotent across order, trade, cash ledger and position; no brokerage execution API.	START: Start after S5-DAT-01 tables and S3-API-01 quote contract; integrate with S5-API-01 when route exists. IF BLOCKED: Implement calculation engine using in- memory objects and fixed test cases first.	EVIDENCE: execution service, transaction boundary design and unit tests. DONE WHEN: Expected cash, average cost, realized and unrealized values match test vectors; failed order changes no ledger state. REVIEW: Palak + Mitali
S5-API-03 Disha	Add security, concurrency and failure tests for paper trading, including double submit, simultaneous orders, missing quote and rollback.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Paper execution is atomic and idempotent across order, trade, cash ledger and position; no brokerage execution API.	START: Start after S5-API-01/02 callable services exist. IF BLOCKED: Prepare concurrency test harness and expected invariants early.	EVIDENCE: integration/load tests and defect report. DONE WHEN: No negative cash/position from tested concurrency; partial writes are absent after forced failure. REVIEW: Payal + Mrugank

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S5-DAT-01 Shivraj	Create paper_accounts, paper_orders, paper_trades, paper_positions, paper_cash_ledger and paper_portfolio_snapshots Delta tables.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Paper ledger records are operational Lakebase transactions; snapshots/performance history are Delta and must reconcile.	START: Start after S1-DAT-01 and paper trading domain model is approved by Syona/Palak. IF BLOCKED: Draft normalized DDL and sample transactions first.	EVIDENCE: data/ddl/040_paper_trading.sql and table notebook. DONE WHEN: Order, trade and ledger identifiers are traceable; cash ledger is append-only; reset is represented without deleting audit history. REVIEW: Mitali + Palak
S5-DAT-02 Pranjal	Build snapshot workflow and deterministic SQL/Python calculations for account equity, cost basis, realized/unrealized P&L and daily return.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Paper ledger records are operational Lakebase transactions; snapshots/performance history are Delta and must reconcile.	START: Start after S5-DAT-01 and usable market quotes from S3-DAT-02. IF BLOCKED: Develop calculations against fixture trades/prices first.	EVIDENCE: Databricks workflow, calculation module and expected- results notebook. DONE WHEN: Results match hand-calculated test portfolios; missing price is flagged, not treated as zero. REVIEW: Mitali + Nayan
S5-DAT-03 Piyush	Create reconciliation notebook comparing orders, trades, positions, cash ledger and snapshots; produce break report.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Paper ledger records are operational Lakebase transactions; snapshots/performance history are Delta and must reconcile.	START: Start after S5-DAT-02 and sample execution data from S5-API-02. IF BLOCKED: Create synthetic good/bad ledgers beforehand.	EVIDENCE: reconciliation notebook, break table and daily check query. DONE WHEN: Injected duplicate/missing ledger entries are identified; clean scenario balances to zero difference. REVIEW: Payal + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S5-AI-01 Likhita	Create paper-trading coach prompt that explains order outcome, portfolio impact, risk and learning points without encouraging real-money execution.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Coaching may explain deterministic paper metrics only and cannot imply real execution, guaranteed returns or fabricated prices.	START: Start after S5-API-02 defines structured trade result and S1-AI-01 safety policy. IF BLOCKED: Use fixed trade fixtures to draft prompt/examples.	EVIDENCE: prompt, response schema and 15 scenario outputs. DONE WHEN: Narrative matches trade values, states simulation status and includes risk/learning explanation. REVIEW: Nayan + Syona
S5-AI-02 Palak	Build performance narrative pipeline using deterministic paper account metrics to summarize contributors, concentration and behavior patterns.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Coaching may explain deterministic paper metrics only and cannot imply real execution, guaranteed returns or fabricated prices.	START: Start after S5-DAT-02 snapshot schema and S5-AI-01 answer format. IF BLOCKED: Develop against saved metric fixtures.	EVIDENCE: AI tool interface, prompt and sample weekly summaries. DONE WHEN: No new numeric calculation by LLM; every stated amount/percentage maps to input field. REVIEW: Pranjal + Mitali
S5-AI-03 Mrugank	Create safety and factuality evaluation for paper-trading coaching: no guaranteed return, no real-trade instruction, exact numeric consistency and disclaimer.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Coaching may explain deterministic paper metrics only and cannot imply real execution, guaranteed returns or fabricated prices.	START: Start after S5-AI-01/02 fixtures are available. IF BLOCKED: Draft adversarial prompts and expected safe behavior first.	EVIDENCE: evaluation set, automated tests and baseline report. DONE WHEN: All numeric mismatches are detected; unsafe prompts produce educational/refusal response. REVIEW: Payal + Syona

Sprint exit gate

- Buy/sell scenarios reconcile across order, trade, position and cash ledger.
- Web/mobile display the same values.
- Simulation disclaimers are present.

6.6 Sprint 6 - Smart alerts and conversational portfolio intelligence

Objective: Deliver alert rules, notifications and portfolio-grounded conversational intelligence.

Syona / BA-UAT lane: Approve alert thresholds/cooldowns and portfolio-intelligence question set; test notification journeys. Evidence: alert/portfolio UAT report.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S6-PRD-01 Likhita	Build React alert management, notification center and portfolio conversation views with rule creation, unread counts and scenario cards.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. In-app notification state is durable; FCM is best-effort. Cooldown/read state and portfolio answers must match on both clients.	START: Start UI from contracts after S6-API-01/02 drafts; integrate when S6-API-03 tests pass. IF BLOCKED: Use fixtures and mock services first.	EVIDENCE: React features, service tests and E2E flow. DONE WHEN: User can create/deactivate alert, mark notifications read and ask portfolio question with displayed data date. REVIEW: Khyati + Syona
S6-PRD-02 Palak	Build Flutter alert management, notification center and conversational portfolio views matching React fields and safety copy.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. In-app notification state is durable; FCM is best-effort. Cooldown/read state and portfolio answers must match on both clients.	START: Start from fixtures after S1-PRD-03; integrate after S6 APIs reach staging. IF BLOCKED: Implement widgets/state management with fake repository first.	EVIDENCE: Flutter modules, tests and emulator demo. DONE WHEN: Push/local-notification foundation is isolated; in-app notification state persists after refresh. REVIEW: Shivraj + Syona
S6-PRD-03 Mrugank	Create cross-platform E2E tests for alert creation/trigger/read and portfolio questions; verify displayed numbers against API response.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. In-app notification state is durable; FCM is best-effort. Cooldown/read state and portfolio answers must match on both clients.	START: Start case design immediately; automation after S6- PRD-01/02 and S6-API-02/03. IF BLOCKED: Build fixture scenarios and expected response maps.	EVIDENCE: E2E suite and reconciliation report. DONE WHEN: All displayed amounts/percentages match API; stale data and unsupported questions show correct limitation. REVIEW: Payal + Mitali

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S6-API-01 Khyati	Implement alert rule CRUD and evaluation endpoints for price, percentage movement, concentration, risk change and goal off-track conditions.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Do not trigger on unknown provider data; enforce cooldown/dedupe and preserve in-app records on FCM failure. Portfolio intelligence consumes accepted snapshot IDs.	START: Start after S6-DAT-01 and S3 market/portfolio services; goal alert may use fixture until S7. IF BLOCKED: Create rule schemas and evaluator unit tests with static inputs first.	EVIDENCE: alerts router/service/evaluator and tests. DONE WHEN: Each rule has explicit units, comparison operator, active status and cooldown; invalid thresholds are rejected. REVIEW: Palak + Syona
S6-API-02 Mitali	Implement deterministic portfolio intelligence service returning allocation, concentration, diversification, top contributors/detractors, volatility input and scenario results.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Do not trigger on unknown provider data; enforce cooldown/dedupe and preserve in-app records on FCM failure. Portfolio intelligence consumes accepted snapshot IDs.	START: Start after S6-DAT-02 metric pipeline and S3 portfolio ownership checks. IF BLOCKED: Build service against fixture snapshots first.	EVIDENCE: portfolio_intelligence service, response schemas and calculation tests. DONE WHEN: All numeric outputs are deterministic and versioned; missing data produces explicit unavailable fields. REVIEW: Nayan + Palak
S6-API-03 Payal	Implement notifications list/read/mark-all-read endpoints and integration tests linking alert event to notification delivery status.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Do not trigger on unknown provider data; enforce cooldown/dedupe and preserve in-app records on FCM failure. Portfolio intelligence consumes accepted snapshot IDs.	START: Start after S6-DAT-01 creates event/notification tables and S6-API-01 emits alert events. IF BLOCKED: Build repository mocks and tests early.	EVIDENCE: notifications router/service and contract tests. DONE WHEN: Unread counts remain consistent; user cannot read another user's notifications; failed delivery remains auditable. REVIEW: Disha + Syona

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S6-DAT-01 Nilisha	Create alert_rules, alert_events, notifications and notification_delivery_attempts tables with indexes/partition strategy and status history.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Rules/events/notifications are operational; portfolio snapshots, alert analytics and run monitoring are Delta. Record trigger source timestamps.	START: Start after S1-DAT-01 and Syona approves alert categories. IF BLOCKED: Draft DDL and sample event lifecycle records first.	EVIDENCE: data/ddl/050_alerts.sql and creation notebook. DONE WHEN: One rule can create many events; delivery attempts are append-only; read status is queryable. REVIEW: Mitali + Palak
S6-DAT-02 Nayan	Build portfolio snapshot/metrics pipeline for allocation, sector exposure, concentration score, diversification score and change from prior snapshot.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Rules/events/notifications are operational; portfolio snapshots, alert analytics and run monitoring are Delta. Record trigger source timestamps.	START: Start after S3 portfolio tables and S5 snapshot IF BLOCKED:	EVIDENCE: Databricks workflow, metrics table and test notebook. DONE WHEN: Metrics are reproducible and linked to snapshot timestamp; totals reconcile with portfolio summary. REVIEW: Mitali + Piyush
S6-DAT-03 Disha	Build alert evaluation job and monitoring queries for due rules, triggered events, duplicate suppression and delivery failures.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Rules/events/notifications are operational; portfolio snapshots, alert analytics and run monitoring are Delta. Record trigger source timestamps.	START: Start after S6-DAT-01/02 and S6-API-01 rule schema. IF BLOCKED: Test job with synthetic rules/events before notification service is ready.	EVIDENCE: scheduled job/notebook, run log and monitoring SQL. DONE WHEN: Same event is not repeatedly emitted within cooldown; failed rows are retryable and visible. REVIEW: Payal + Palak

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S6-AI-01 Shivraj	Build portfolio-question intent router for holdings, allocation, concentration, performance, risk, scenario and unsupported requests.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Ground answers in authorized immutable snapshots and deterministic metrics; test stale/missing price, concentration scenarios and cross-user isolation.	START: Start after S6-API-02 response schema and S1-AI-01 policy. IF BLOCKED: Create labeled intent dataset and baseline rule classifier first.	EVIDENCE: intent router, dataset and accuracy report. DONE WHEN: Test accuracy meets agreed threshold; unsupported questions route to education or safe limitation. REVIEW: Nayan + Syona
S6-AI-02 Pranjal	Build conversational portfolio explanation and optimization prompt/tool chain using deterministic metrics, user risk profile and educational recommendation constraints.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Ground answers in authorized immutable snapshots and deterministic metrics; test stale/missing price, concentration scenarios and cross-user isolation.	START: Start after S6-API-02, S2 risk profile and S6-AI-01 IF BLOCKED:	EVIDENCE: tool schemas, prompts and scenario examples. DONE WHEN: Recommendations explain current versus suggested allocation, assumptions and risk; no automatic trade action is offered. REVIEW: Nayan + Palak
S6-AI-03 Piyush	Create scenario/risk evaluation set for concentration, 15% sector decline, missing price, conservative user and conflicting goal/risk cases.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Ground answers in authorized immutable snapshots and deterministic metrics; test stale/missing price, concentration scenarios and cross-user isolation.	START: Start after S6-AI-02 response format. IF BLOCKED: Draft expected facts and prohibited outputs first.	EVIDENCE: evaluation dataset and regression report. DONE WHEN: Numeric consistency, profile alignment, disclaimer and recommendation transparency are scored separately. REVIEW: Mrugank + Syona

Sprint exit gate

- Alerts trigger without duplicate spam.
- Portfolio answers use deterministic metrics.
- Notification read state works on both clients.

6.7 Sprint 7 - Goals and spending trend analysis

Objective: Deliver goal planning and privacy-controlled spending trend MVP.

Syona / BA-UAT lane: Approve goal assumptions, CSV privacy wording, synthetic transaction file and category correction scenarios. Evidence: privacy/UAT evidence.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S7-PRD-01 Shivraj	Build React goal creation/projection/progress and spending CSV upload/dashboard screens with assumption, privacy, loading and correction states.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. CSV/manual spending is the MVP; preview errors before commit, mask sensitive descriptions before AI and label goal assumptions.	START: Start layout from approved stories; integrate after S7- API-01/02 contract tests. IF BLOCKED: Use static goal/transaction fixtures and local file parsing mock first.	EVIDENCE: React feature modules, services, tests and screenshots. DONE WHEN: User can create/edit goal, compare scenarios, upload CSV, correct categories and view monthly trends. REVIEW: Khyati + Syona
S7-PRD-02 Pranjal	Build Flutter goal and spending screens with file picker abstraction, progress charts and category correction flow.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. CSV/manual spending is the MVP; preview errors before commit, mask sensitive descriptions before AI and label goal assumptions.	START: Start from fixtures after S1-PRD-02; integrate after S7 APIs are staging-ready. IF BLOCKED: Use bundled synthetic CSV and mock repository.	EVIDENCE: Flutter feature modules, tests and emulator evidence. DONE WHEN: Same validation/assumption/privacy wording as React; file REVIEW:
S7-PRD-03 Piyush	Create cross-platform upload/projection tests, accessible chart text summaries and analytics events for goal/spending flows.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. CSV/manual spending is the MVP; preview errors before commit, mask sensitive descriptions before AI and label goal assumptions.	START: Start test design from stories; automate after S7-PRD- 01/02 stable test IDs. IF BLOCKED: Prepare fixtures for valid, duplicate, malformed and sensitive-description files.	EVIDENCE: React/Flutter tests, chart summaries and event evidence. DONE WHEN: No raw transaction description appears in analytics; displayed totals reconcile with API summaries. REVIEW: Payal + Mitali

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S7-API-01 Likhita	Implement goal CRUD, projection and progress endpoints using deterministic projection module with explicit assumptions.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Version deterministic goal assumptions. Spending upload supports preview/commit, correction persistence, duplicate detection and user isolation.	START: Start after S7-DAT-01; projection formula tests can begin with in-memory models. IF BLOCKED: Build schemas/calculation tests before repository integration.	EVIDENCE: goals router/service/repository, calculation module and tests. DONE WHEN: Invalid dates/amounts are rejected; projection includes assumptions and gap; ownership enforced. REVIEW: Palak + Mitali
S7-API-02 Palak	Implement transaction CSV upload, validation, masked storage, category review/update and monthly summary endpoints.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Version deterministic goal assumptions. Spending upload supports preview/commit, correction persistence, duplicate detection and user isolation.	START: Start after S7-DAT-02 schema and privacy rules; IF BLOCKED:	EVIDENCE: spending router/service, upload parser and tests. DONE WHEN: Malformed files return row-level errors; duplicate upload is identified; raw descriptions are protected. REVIEW: Disha + Syona
S7-API-03 Mrugank	Create privacy, authorization and integration tests for goals/spending; verify file size/type limits, masking, user isolation and summary reconciliation.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Version deterministic goal assumptions. Spending upload supports preview/commit, correction persistence, duplicate detection and user isolation.	START: Start after S7-API-01/02 expose stable routes. IF BLOCKED: Prepare threat cases and synthetic files early.	EVIDENCE: integration/security tests and privacy verification report. DONE WHEN: Cross-user access fails; disallowed files are rejected; response/logs do not expose raw sensitive content. REVIEW: Payal + Syona

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S7-DAT-01 Khyati	Create financial_goals, goal_contributions, goal_projections and goal_progress_snapshots Delta tables with versioned projection assumptions.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Goals/uploads/transactions are operational; summaries/trends/snapshots are Delta. Reconcile accepted rows and preserve correction history.	START: Start after S1-DAT-01 and Syona approves goal fields/assumption wording. IF BLOCKED: Draft DDL and example goal scenarios first.	EVIDENCE: data/ddl/060_goals.sql and creation notebook. DONE WHEN: Projection assumptions are stored with each result; contribution history is append-only; goal ownership is explicit. REVIEW: Mitali + Palak
S7-DAT-02 Mitali	Create transaction_uploads, transactions, transaction_categories, monthly_spending_summaries and spending_insights tables; build CSV normalization and masking pipeline.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Goals/uploads/transactions are operational; summaries/trends/snapshots are Delta. Reconcile accepted rows and preserve correction history.	START: Start after privacy fields and supported CSV format are approved by Syona/Palak. IF BLOCKED: Create synthetic CSV files and parser locally first.	EVIDENCE: data/ddl/061_spending.sql, ingestion notebook and quarantine table. DONE WHEN: Duplicate files/rows are detected; raw descriptions are masked for AI use; invalid rows are quarantined with reason. REVIEW: Palak + Syona
S7-DAT-03 Payal	Build goal/spending data quality and analytics checks: target/date validity, contribution reconciliation, category totals, duplicate transactions and monthly summary reconciliation.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Goals/uploads/transactions are operational; summaries/trends/snapshots are Delta. Reconcile accepted rows and preserve correction history.	START: Start after S7-DAT-01/02. IF BLOCKED: Prepare SQL rules and synthetic failures first.	EVIDENCE: DQ notebook, monthly summary query and failure report. DONE WHEN: Monthly category totals equal transaction totals; injected duplicates/invalid goals are detected. REVIEW: Piyush + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S7-AI-01 Nilisha	Create goal-planning explanation tool that interprets deterministic projection, required monthly contribution, shortfall and assumptions in simple language.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. No guaranteed outcomes. Categorization uses masked text, exposes confidence, accepts corrections and has a rules/deterministic fallback.	START: Start after S7-API-01 projection schema or agreed fixture and S1-AI-01 safety policy. IF BLOCKED: Draft prompts/examples using fixed projection JSON.	EVIDENCE: tool schema, prompt and 12 scenario outputs. DONE WHEN: Narrative does not guarantee target achievement; all amounts and dates match input. REVIEW: Nayan + Syona
S7-AI-02 Nayan	Build transaction categorization v1 using rules plus an ML/LLM fallback on masked descriptions; return category and confidence.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. No guaranteed outcomes. Categorization uses masked text, exposes confidence, accepts corrections and has a rules/deterministic fallback.	START: Start after S7-DAT-02 publishes masked transaction schema and approved categories. IF BLOCKED: Train/test with synthetic labeled transactions first.	EVIDENCE: categorization module/notebook, labeled dataset and evaluation report. DONE WHEN: Rules are deterministic; low-confidence items are marked for user review; raw sensitive descriptions are not sent externally. REVIEW: Mitali + Palak
S7-AI-03 Disha	Build spending trend and anomaly evaluation: recurring expense, month-over-month increase, unusual transaction and savings-rate explanation cases.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. No guaranteed outcomes. Categorization uses masked text, exposes confidence, accepts corrections and has a rules/deterministic fallback.	START: Start after S7-AI-02 outputs categories and S7-DAT-03 summaries. IF BLOCKED: Create labeled expected cases in parallel.	EVIDENCE: evaluation dataset, scoring script extension and report. DONE WHEN: False-positive/false-negative examples are listed; insights use masked/category-level data. REVIEW: Mrugank + Syona

Sprint exit gate

- Goal projections expose assumptions.
- CSV spending flow protects descriptions and reconciles totals.
- AI categorization exposes confidence and correction.

6.8 Sprint 8 - Production hardening and release

Objective: Harden, evaluate, secure and release the integrated platform.

Syona / BA-UAT lane: Coordinate full release UAT, exception decisions, help content, known limitations and release sign-off. Evidence: signed release checklist and backlog.

Sprint cadence: Day 1 Ready/spec review; Day 2 scaffolds/migrations/generated clients/fixtures; Days 3-6 implementation; Day 7 peer/consumer review; Day 8 integration; Day 9 DQ/AI/security/accessibility/UAT; Day 10 demo/handoff/retrospective.

Product track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S8-PRD-01 Nilisha	Harden React application: accessibility, route-level loading, bundle size, error monitoring, production environment, regression and deployment runbook.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Release React to Azure Static Web Apps and Android APK through GitHub Releases; keep iOS-compatible source/simulator build and document free-tier limits.	START: Start after feature freeze at end of S7 and staging API availability. IF BLOCKED: Run audits and prepare issue list before freeze.	EVIDENCE: release PR, accessibility/performance reports, production build and runbook. DONE WHEN: Zero release-blocking accessibility defect; no localhost/secrets in bundle; rollback steps are tested. REVIEW: Khyati + Syona
S8-PRD-02 Nayan	Harden Flutter application: accessibility, performance, secure storage review, Android release build, iOS archive configuration, crash monitoring and store packet.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Release React to Azure Static Web Apps and Android APK through GitHub Releases; keep iOS-compatible source/simulator build and document free-tier limits.	START: Start after feature freeze and staging environment values. IF BLOCKED: Prepare signing/config/checklists without production secrets.	EVIDENCE: release build artifacts, test report, store metadata draft and runbook. DONE WHEN: Android build installs and completes smoke flow; iOS configuration is reproducible; no secrets/logged tokens. REVIEW: Shivraj + Syona
S8-PRD-03 Disha	Coordinate cross-platform UAT fixes, verify shared copy/validation/data values and produce final user help/known-limitations content.	Use generated TypeScript/Dart clients from frozen OpenAPI; no direct response-shape assumptions or client-side financial math. Implement required loading/empty/error/disabled/success states plus accessibility and web/mobile parity. Release React to Azure Static Web Apps and Android APK through GitHub Releases; keep iOS-compatible source/simulator build and document free-tier limits.	START: Start after S8-PRD-01/02 release candidates and Syona UAT cases. IF BLOCKED: Prepare parity checklist and help outline early.	EVIDENCE: fixed PRs, parity report, help content and limitations list. DONE WHEN: React and Flutter display same business results; all accepted limitations have backlog ticket. REVIEW: Payal + Syona

API track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S8-API-01 Shivraj	Harden FastAPI security: rate limits, CORS, token rotation/revocation, authorization tests, request size limits and secure headers.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Deploy Dockerized FastAPI to Container Apps with secrets, security controls, observability and rollback; readiness reflects dependency health.	START: Start after S7 API feature freeze and S2 auth foundation. IF BLOCKED: Run threat checklist and create failing security tests first.	EVIDENCE: security PR, test evidence and configuration guide. DONE WHEN: Critical authorization tests pass; excessive/invalid requests are safely rejected and audited. REVIEW: Palak + Payal
S8-API-02 Pranjal	Add observability and deployment: structured logs, metrics, tracing, health/readiness dependencies, staging/prod config and rollback procedure.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Deploy Dockerized FastAPI to Container Apps with secrets, security controls, observability and rollback; readiness reflects dependency health.	START: Start after core services are stable; tracing fields from S4-API-03 must exist. IF BLOCKED: Define logging schema and deployment checklist early.	EVIDENCE: observability middleware, dashboards/queries, deployment and rollback runbooks. DONE WHEN: A test request can be traced through API, data and AI; failed dependency makes readiness fail but liveness remain healthy. REVIEW: Palak + Mitali
S8-API-03 Piyush	Run API integration/load/reliability tests for login, portfolio, chat, paper trading, alerts, goals and spending; resolve bottlenecks assigned.	Use router -> service -> repository/port separation. Return standard correlation/data-freshness metadata, enforce authorization, and use provider adapters rather than vendor SDKs in domain services; add unit/contract/integration tests. Deploy Dockerized FastAPI to Container Apps with secrets, security controls, observability and rollback; readiness reflects dependency health.	START: Start after S8-API-01/02 on staging and stable seed/reset scripts. IF BLOCKED: Prepare load scenarios and performance budgets beforehand.	EVIDENCE: load scripts, results, fixed PRs and capacity note. DONE WHEN: No data corruption under test; critical p95/timeout goals are met or documented with accepted mitigation. REVIEW: Disha + Syona

Data track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S8-DAT-01 Likhita	Implement Unity Catalog permissions and environment separation for identity, portfolio, AI, trading and spending data; validate least privilege.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Verify least privilege, logical environment isolation, retention/export/delete, workflow retries and final reconciliation; document paid-tier separation.	START: Start after all S2-S7 tables exist and service identities are known. IF BLOCKED: Draft access matrix and SQL grants before execution.	EVIDENCE: access-control script, matrix and validation evidence. DONE WHEN: Web/mobile never access Databricks directly; service identities can access only required schemas; denied tests pass. REVIEW: Mitali + Palak
S8-DAT-02 Palak	Orchestrate production Databricks workflows for market ingestion, RAG ingestion, portfolio snapshots, alert evaluation, paper snapshots and spending summaries with retries/alerts.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Verify least privilege, logical environment isolation, retention/export/delete, workflow retries and final reconciliation; document paid-tier separation.	START: Start after each source notebook/job passes its sprint tests and S8-DAT-01 permissions are defined. IF BLOCKED: Create dependency DAG and failure- handling plan first.	EVIDENCE: workflow definitions, schedule matrix, retry policy and failed-run demo. DONE WHEN: Upstream failure prevents unsafe downstream execution; retries are bounded; owners receive actionable failure context. REVIEW: Mitali + Syona
S8-DAT-03 Mrugank	Run final data reconciliation, retention, backup/export and deletion verification across user, portfolio, trading, AI and spending domains.	Operational/user-owned records use Lakebase/Postgres + Alembic; Delta/SQL is for history, analytics, snapshots/events/evaluation. Use UUID/UTC conventions, DQ, lineage, user isolation and rollback/recovery evidence. Verify least privilege, logical environment isolation, retention/export/delete, workflow retries and final reconciliation; document paid-tier separation.	START: Start after S8-DAT-01/02 and final staging dataset. IF BLOCKED: Prepare reconciliation rules and deletion scenarios early.	EVIDENCE: final data quality report, retention/deletion matrix and recovery rehearsal evidence. DONE WHEN: Critical DQ checks pass; account deletion/export behavior is verified; recovery steps are documented. REVIEW: Piyush + Syona

AI/ML track

Task / owner	What to implement	Scale-ready requirements	Start / while blocked	Evidence / acceptance / review
S8-AI-01 Khyati	Run full AI regression across education, risk, comparison, RAG, paper trading, portfolio, goals and spending; fix prompt/tool failures assigned to the cohort.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Register approved versions/evidence in MLflow where available; finish regression, red-team, monitoring, rollback targets and quota-exhaustion fallback.	START: Start when S2-S7 AI modules are versioned and deployed to staging. IF BLOCKED: Consolidate datasets and expected thresholds before deployment completes.	EVIDENCE: master evaluation suite, regression report and fixed prompt/tool PRs. DONE WHEN: No critical safety failure; thresholds are met or explicitly accepted with owner and mitigation. REVIEW: Nayan + Syona
S8-AI-02 Mitali	Register AI experiments/models/prompts in MLflow, define version tags, promote approved versions and create latency/error/quality monitoring queries.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Register approved versions/evidence in MLflow where available; finish regression, red-team, monitoring, rollback targets and quota-exhaustion fallback.	START: Start after S8-AI-01 identifies candidate versions and model serving access exists. IF BLOCKED: Prepare experiment naming, tags and monitoring schema early.	EVIDENCE: MLflow records/model registry entries, promotion checklist and monitoring dashboard queries. DONE WHEN: Every production AI component has owner, version, dataset, metrics and rollback target. REVIEW: Nayan + Palak
S8-AI-03 Payal	Execute red-team tests for prompt injection, unsupported advice, stale data, fabricated citation, privacy leakage and cross-user portfolio requests.	Version prompts and JSON schemas; FastAPI validates structured output. Deterministic calculations stay outside the LLM. Store model/prompt/retriever versions and test stale, unsafe, cross-user, malformed and insufficient-evidence cases. Register approved versions/evidence in MLflow where available; finish regression, red-team, monitoring, rollback targets and quota-exhaustion fallback.	START: Start after production candidate AI/API stack is on staging. IF BLOCKED: Prepare adversarial prompts and expected safe outcomes beforehand.	EVIDENCE: red-team report, failing-case tickets and retest evidence. DONE WHEN: All critical attacks are blocked or safely limited; logs contain no secret or other-user data. REVIEW: Mrugank + Syona

Sprint exit gate

- Critical UAT/security/data/AI checks pass.
- Production builds and rollback procedures are reproducible.
- All exceptions have owner and accepted mitigation.

7. Handoff, blocker and AI-assisted development rules

Dependency handoff

- A handoff is valid only when the owner posts task ID, merged commit/PR, contract/schema or sample payload, test command, seed/fixture location, known limitations and reviewer/consumer acceptance.
- A verbal statement such as “the API is ready” is not a handoff.

Blocker escalation

Blocked time	Required action
< 4 working hours	Use listed parallel work/mocks/tests and contact predecessor owner in the task thread.
4-8 working hours	Raise in daily stand-up; predecessor and dependent owner pair for 30 minutes.
> 1 working day	Syona records blocker; sprint lead reorders work or approves a temporary mock contract.
> 2 working days	Split, reassign or move scope; no developer remains idle waiting for another track.

AI-assisted development PR evidence

- Problem and contract: user story, task ID, input/output contract and dependencies.
- AI usage: tool used, prompt summary, generated areas and rejected suggestions.
- Human verification: explain code flow, security/privacy implications and at least one manual improvement.
- Tests: commands run plus positive/negative cases and evidence.
- Data/AI impact: tables/events/model/prompt versions, PII handling and rollback.
- Review: cohort/track peer plus technical steward; Syona for user-facing acceptance.

8. Non-negotiable product and free-tier guardrails

- The free edition is an educational prototype, not a regulated production environment or commercial SLA-backed service.
- Market information always carries explicit freshness/data status and is never described as guaranteed real-time unless the provider contract supports that claim.
- Paper trading never moves real money and cannot call a brokerage execution API.
- AI output is educational decision support; deterministic calculations belong in backend services and the LLM explains rather than invents values.
- Direct bank aggregation is deferred; spending MVP uses manual entry and CSV upload.
- Public App Store / Play Store publication and custom domain are outside the zero-cost phase.
- Paid production requires physically separate environments and stronger network/security controls; free-tier logical schemas/revisions are not treated as production isolation.

9. Source synthesis

Execution source: FinEx Equal-Opportunity Rotation Execution Handbook v4 - used for the eight-sprint sequence, S1-PRD/API/DAT/AI task IDs, owners, track rotation, reviewers, blockers, exit gates and BA/UAT lane.

Engineering source: FinEx Specification-First Implementation and Deployment Handbook v5 - used for the free-tier architecture, Lakebase/Delta split, repository/contracts, provider boundaries, OpenAPI/generated clients, environment/deployment model, Definition of Ready/Done, detailed safety/privacy/testing rules and paid-tier migration constraints.